SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Praveen J (1921260144) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

I. Greedy Best-First Search

Greedy Best-First Search (GBFS) chooses the next location that appears closest to the destination, based on the heuristic estimate \(h(n)\), such as straight-line distance. It does not directly account for the cost already spent reaching that location.

Strengths

- Often finds a route quickly because it heads toward the destination.
- Simple to implement and useful when decisions must be made fast.

Weaknesses under uncertainty

- A route that looks short may pass through flooded or dangerous areas.
- Straight-line distance does not reflect changing weather, terrain, or travel costs.
- It can choose a poor route or get stuck pursuing an inaccessible path.
- It does not guarantee the shortest or safest route.

II. A* Search

A* evaluates each possible location using:

\[f(n) = g(n) + h(n) \]

Here, \(g(n)\) is the cost so far, including travel time or risk, and \(h(n)\) estimates the remaining cost to the destination. A* considers both the route already taken and the estimated distance left.

If the heuristic never overestimates the true remaining cost, and the cost information stays accurate, A* can find an optimal route according to the costs it was given. Those costs should include relevant factors such as weather, terrain, and safety risk.

III. Impact of Heuristic Accuracy

A good heuristic helps both algorithms prioritize promising routes. Straight-line distance is easy to calculate, but may be misleading when terrain, weather, or blocked paths affect actual travel.

- **GBFS depends heavily on the heuristic:** an inaccurate estimate can send it toward a seemingly nearby but costly or unsafe route.
- *A is less dependent on the heuristic alone:** it also considers the cost already incurred, \(g(n)\). A more accurate heuristic can make A* faster, while a poor one can make it explore more locations.
- For A* to retain its optimality guarantee, its heuristic must not overestimate the remaining cost.

IV. Effect of Dynamic Changes

When a road or flight path becomes blocked, the drone's existing route may no longer be usable. Weather changes can also alter travel time and risk.

The system should receive updates, mark blocked paths as unavailable, revise affected costs, and replan from the drone's current position. Frequent changes can require repeated searches and increase delay. Searching with current map and weather data helps, but no algorithm can guarantee a safe route if its information is outdated.

V. Recommendation

*A is the stronger choice when the drone needs to balance delay, risk, and route quality**, because it considers both the cost already travelled and the estimated cost remaining. GBFS may be faster to produce an initial route, but it can overlook expensive or dangerous sections because it focuses on the heuristic.

For emergency delivery, use A* with up-to-date blockage and weather information, and include risk in the route costs. If time is extremely limited, the system can use a fast initial route and replan as updates arrive. Safety checks and the ability to reject unsafe paths should remain part of the system; neither search algorithm can ensure safety on its own.

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Optimization problem formulation

The system can treat each traffic signal's timings—such as green-light duration, cycle length, and coordination with nearby signals—as decision variables. A **state** is one complete timing plan across the city. The system evaluates each plan using a cost function that combines:

- Total vehicle waiting time
- Fuel consumption, including fuel wasted while idling
- Traffic congestion, such as queue lengths and travel delays

The goal is to find a timing plan that minimizes the combined cost while respecting constraints, such as safe minimum green times and pedestrian crossing periods. The system must keep updating its plan as traffic conditions change.

Traditional search methods can be inefficient because the number of possible plans grows rapidly with the number of intersections and timing choices. Evaluating each plan may also require traffic simulation or live data. Exhaustively checking every possible plan is therefore impractical, especially when traffic conditions keep changing.

I. Hill Climbing

Hill Climbing starts with a timing plan, tests nearby alternatives, and moves to one that improves the cost. For example, it might lengthen a busy road's green phase or adjust a group of coordinated signals. It repeats this process until none of the tested nearby changes improves the plan.

It is relatively simple and can make quick improvements, but it only considers nearby alternatives. It may stop before finding a better overall plan.

II. Local maxima, plateaus, and ridges

These are search problems; for this minimization task, they correspond to getting stuck at a locally best plan or having difficulty finding progress.

- **Local optimum:** A plan is better than its immediate alternatives, but a substantially better plan exists elsewhere. For example, improving one intersection at a time may fail to reveal that several neighboring signals need to change together.

- **Plateau:** Many nearby plans have almost the same cost. The search has no clear improvement direction, even though a better plan may be farther away. This can happen when small timing changes have little measurable effect.
- **Ridge:** Progress requires a coordinated series of changes. Changing only one signal may make traffic worse temporarily, while changing several connected signals together would reduce queues. Hill Climbing may fail to make that first worsening step.

III. Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA) sometimes accepts a change that temporarily worsens the plan. Early in the search, it is more willing to do this, which can help it escape a local optimum or move across a plateau. Over time, it becomes less willing to accept worse changes and focuses on refining the solution. It can still struggle if the traffic model changes faster than it can search.

Genetic Algorithms (GA) maintain a population of timing plans. They select better-performing plans, combine parts of them, and introduce random changes to create new candidates. Combining plans can share useful timings across intersections, while random changes help explore alternatives. GAs can search many regions at once, but evaluating candidates can be costly and results depend on choices such as population size and mutation rate.

Recommendation

A Genetic Algorithm is a good fit for city-wide coordination because it explores many timing plans at once and can combine successful choices across intersections. This is useful when the search space is large and good signal timings depend on neighboring intersections.

To adapt to changing traffic, the system should repeatedly evaluate plans using fresh traffic data, keep a good current plan, and search for improvements in the background. Simulated Annealing can be a simpler alternative when the number of adjustable variables is more limited. In either case, the system should enforce safety and pedestrian constraints, and test candidate plans before applying them city-wide.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search
ii. Analyze the challenges of partial observability and dynamic environments
iii. Describe how the rover builds and updates its internal model
iv. Compare online search with uninformed and informed search strategies
v. Justify the most suitable approach considering uncertainty, adaptability, and safety

Mars Rover Navigation

I. Why an online search agent is needed

Offline search assumes the agent knows the full map and action outcomes before it starts planning. A Mars rover has only a partial map and limited sensing. It must move, gather new information, and update its route as it discovers rocks, craters, and terrain conditions.

An **online search agent** plans while operating: it chooses an action from what it currently knows, observes the result, then replans. This lets it explore unknown terrain and respond when its initial assumptions prove wrong.

II. Partial observability and changing conditions

The rover's sensors may detect only nearby hazards, and measurements may be incomplete or noisy. A hazard outside sensor range can make a planned path unsafe. Terrain may also affect movement differently than expected, such as slowing the rover or making a route impassable.

Because the rover cannot rely on a complete map, it must account for uncertainty and avoid treating unobserved terrain as confirmed safe. Communication delays also mean it may need to make decisions without waiting for instructions from Earth.

III. Building and updating an internal model

The rover maintains an internal map or **world model** containing known safe areas, hazards, terrain types, and unexplored regions. It updates this model as it receives sensor readings and observes the results of its movements.

For example, if an area appears rocky, the rover can mark it as difficult or risky. If movement takes longer than expected, it can revise its estimate of travel cost. It then uses the updated model to select or revise its next route.

IV. Online search compared with uninformed and informed search

- **Online search** gathers information through action and observation. It suits unknown environments because it can revise its plan as new information arrives.
- **Uninformed search** explores without using estimates of how close a location is to a goal. In a large terrain map, this can spend considerable time exploring unpromising routes.
- **Informed search** uses a heuristic, such as estimated distance to a sample site. It can guide exploration more efficiently, but its estimates may be inaccurate when terrain is unknown.

Online search describes *when and how* the rover plans; uninformed and informed search describe *how* it chooses among options. An online rover can use informed methods, provided it updates its estimates as it explores.

V. Recommended approach

Use an **online, informed search approach** with frequent sensing, replanning, and conservative safety rules. A heuristic such as distance to the sample site can help guide the rover, while the

internal map records uncertainty and risk. The rover should prefer routes with known safe terrain when alternatives are similar and stop or replan when sensor data indicates a hazard.

This approach adapts to incomplete information and avoids the cost of requiring a full map in advance. However, no search method can guarantee safety when sensors miss hazards, so safety margins and the ability to pause are essential.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation
ii. Explain how backtracking search works in this scenario
iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
iv. Analyze real-world challenges and justify the best strategy for scalability

Exam Timetabling as a Constraint Satisfaction Problem

A Constraint Satisfaction Problem (CSP) consists of **variables**, their **domains**, and **constraints** that define which combinations are allowed. The goal is to assign a time and room to every exam while satisfying all hard constraints.

I. Variables, domains, and constraints

Variables: Create one variable for each course exam. Its value represents an assigned time slot and exam hall, for example, *Tuesday morning in Hall A*.

Domains: Each exam's domain contains the feasible time-slot and hall combinations. Remove options in advance if the hall is too small, unavailable, or lacks required accessibility features.

Constraints:

- **No student conflicts:** Exams taken by the same student cannot overlap.
- **Hall capacity:** A hall must have enough seats for the students taking the exam.
- **Limited halls:** No more exams can be assigned to a time slot than the available halls can accommodate.
- **Subject ordering:** If one subject must be held before another, their time slots must follow that order.
- **Faculty availability:** An exam must be scheduled when its required faculty members are available.

- **Accommodations:** Students who need extra time or special facilities must have a suitable room and schedule. Their exam duration may also affect room availability.

II. Backtracking search

Backtracking assigns values to exam variables one at a time. It selects an exam, chooses a feasible time and hall, and checks whether that choice conflicts with assignments already made. If it does, the system tries another value. If no value works, it undoes an earlier assignment and tries a different choice.

For example, if assigning two exams to the same slot creates a student conflict, the system rejects that choice and considers another slot or hall. Backtracking can find a valid timetable, but may take a long time if it explores many combinations that later fail.

III. Constraint propagation

Constraint propagation reduces the search by removing choices that cannot work:

- **Forward checking:** After assigning an exam, remove conflicting slots or halls from the domains of related exams. If any exam has no valid options left, backtrack immediately.
- **Constraint prioritization:** Schedule the most constrained exams first, such as exams with few available slots or many shared students.
- **Value ordering:** Try the option that leaves the most choices open for other exams.

These methods detect conflicts early, so the search avoids exploring many doomed assignments.

IV. Real-world challenges and scalable strategy

Real schedules may involve thousands of students and many courses. Enrolment changes, room availability, faculty schedules, accessibility needs, and exam durations all add constraints. Some preferences—such as avoiding late exams—are desirable but not mandatory, so the system must distinguish hard constraints from soft preferences.

A practical strategy is **backtracking with constraint propagation**, choosing the most constrained exams first. For a large university, the system can schedule related groups in stages, then refine the timetable to improve soft preferences such as minimizing student gaps or avoiding consecutive exams. This approach preserves required constraints while reducing unnecessary search.

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Minimax and Alpha-Beta Pruning for a Strategy Game

I. How Minimax models the game

Minimax represents possible moves as a game tree. Each node is a game state, and each branch is a legal move. It assumes two players take turns:

- At the AI's turn, it chooses the move with the highest value (**maximizing**).
- At the opponent's turn, it assumes they choose the move that gives the AI the lowest value (**minimizing**).

The algorithm explores possible moves to a chosen depth, scores the resulting states, then works backward through the tree to select the AI's best move, assuming the opponent also plays well.

II. Computational challenges

If each state has \(b\) possible moves and the search goes \(d\) moves deep, Minimax may examine roughly \(b^d\) states. In a real-time strategy game, many units and actions can create a very large number of possible moves. Exploring the entire tree may take longer than the decision deadline, and deeper search can improve play but costs more computation.

III. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning gives the same result as Minimax at the chosen search depth but skips branches that cannot affect the final decision.

It tracks:

- **Alpha (\(\alpha\))**: the best value the maximizing player can guarantee so far.
- **Beta (\(\beta\))**: the best value the minimizing player can guarantee so far.

When a branch is already worse than an option found earlier, the algorithm stops exploring it. For example, if the opponent has a move that limits the AI's result to 3, there is no need to explore another line that can only give the AI 5 or more if the AI already has a safer choice worth 6. The opponent would choose the line that limits the AI's result.

Good move ordering helps Alpha-Beta find strong options earlier, so it can prune more of the tree. In the best case, this allows a much deeper search than plain Minimax within the same time.

IV. Evaluation function

When the search reaches its depth limit, an evaluation function estimates how favorable the state is for the AI. It could combine:

\[\text{Score} = w_1(\text{resources})+ w_2(\text{unit strength})+ w_3(\text{map control})+ w_4(\text{objectives})- w_5(\text{exposure to threats}) \]

Factors might include:

- Available resources and production capacity
- Number, strength, and health of units
- Control of important territory or objectives
- Defensive position and vulnerability to attacks
- Progress toward victory conditions

The weights \(w_1,\ldots,w_5\) reflect how important each factor is. They should change with the game stage: resource gathering may matter early, while immediate threats and objectives may matter more near the end. The evaluation should also avoid double-counting related factors and be tested against varied play.

V. Balancing decision quality and speed

Use **depth-limited Minimax with Alpha-Beta pruning**, prioritizing promising moves first. Under a strict deadline, iterative deepening can search depth by depth and keep the best move found so far if time runs out. A tuned evaluation function helps judge states beyond the searched depth.

This approach gives strong decisions without requiring the AI to explore the full game tree. Its answer is optimal only within the searched tree and under the evaluation function, so search depth and evaluation quality affect play.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand